



EDITORA AGÊNTICA

PLAYBOOK — HARNESS ENGINEERING — DO MODELO AO SISTEMA AUTÔNOMO CONFIÁVEL

PARA INICIANTES

Heverton Eduardo Peres

ESPECIALISTA EM MARKETING E DESENVOLVIMENTO DE SOLUÇÕES

Objetivo do Material

O leitor é um escalador que começa no chão da parede do mundo agêntico. Cada harness é um equipamento de escalada — corda, cinto, mosquetão, capacete — que dá alavancagem (fazer mais com o modelo) e proteção (não cair quando algo falha). A cada capítulo ele sobe um trecho da rota, e só alcança o cume quando todos os equipamentos estão instalados e testados.

Obra-mãe: harness-engineering

Passos práticos: 8

Você é: Escalador de Harnesses

Sumário

Objetivo do Material	4
Como usar este playbook	5
Mapa dos Estágios	6
Passos Práticos	7
Passo 1 — A Revolução dos Agentes: Por Que o Modelo Não Basta	7
Passo 2 — Anatomia de um Harness: O Corpo Que Carrega o Cérebro	8
Passo 3 — Test Harness: A Herança da Engenharia de Software	10
Passo 4 — Safety Harness e Guardrails: A Camada Que Impede a Queda	11
Passo 5 — O Ciclo ReAct e os Loops de Execução	12
Passo 6 — Sandboxes, Permissões e o Controle de Execução	14
Passo 7 — Gestão de Contexto: Combatendo o Context Rot	15
Passo 8 — Harness em Produção: Observabilidade, Evals e o Engenheiro Agêntico	16
Checklist Mestre	19

Objetivo do Material

Introdução de impacto: o dia em que o modelo errou sozinho — e ninguém tinha um arnês. Apresenta a máxima Agente = Modelo + Harness e o que o leitor será capaz de construir ao final da obra.

Como usar este playbook

Você é o **Escalador de Harnesses**. Cada passo é um card independente com sete partes: objetivo, pré-requisito, entregas, execução, gate de verificação, critério de conclusão e armadilhas.

Este documento **não repete a teoria** do livro. Quando precisar do porquê, siga a referência cruzada do card para o capítulo correspondente.

Mapa dos Estágios

#	Estágio	Passos
1	Arnês	1, 2, 3, 4
2	Corda	5, 6, 7, 8

Passos Práticos

Passo 1 — A Revolução dos Agentes: Por Que o Modelo Não Basta

Estágio: Arnês · **Origem:** Cap. 1 — A Revolução dos Agentes: Por Que o Modelo Não Basta

① Objetivo do passo

Explicar por que LLMs sozinhos não produzem trabalho confiável e introduzir a equação $\text{Agente} = \text{Modelo} + \text{Harness}$, com o mapa do que será construído na obra.

② Pré-requisito

Nenhum — este é o ponto de partida

③ Entregas

- AGENTS.md

④ Execução

Explicar por que LLMs sozinhos não produzem trabal

```
"""Harness minimo: ferramenta + teste + limite."""

class LLM:
    def responder(self, p: str) -> str:
        return "4" # plausivel, porem errado p/ 2+2

class Harness:
    def __init__(self, m): self.m, self.t = m, 0
    def executar(self, p):
        while self.t < 3:
            self.t += 1
            r = self.m.responder(p)
            if self._testar(r, p):
                return r
            raise RuntimeError("limite excedido")
    def _testar(self, r, p):
        return r.strip() == "4" if "2+2" in p else bool(r)

print(Harness(LLM()).executar("Quanto e 2+2?"))
```

⑤ Verificação / Gate

```
python scripts/validar-codigo.py livros/harness-engineering --capitulo 1 --executar
```

⑥ Feito quando...

- ☐ Exercício 1 — Inventário do arnês.** Liste os cinco componentes de um harness (âncora
- ☐ Escreva uma frase dizendo qual falha do agente ele evita
- ☐ Use uma tabela como a abaixo
- ☐ Complete a função `executar_acao` para que o harness valide a ação antes de entregá-la ao modelo — a lição central do capítulo: quem executa é o harness
- ☐ Exercício 3 — Diagnóstico.** Um agente de suporte apagou um arquivo de produção porque o prompt do sistema dizia “você tem autonomia total”
- ☐ Aponte: (a) qual componente do harness deveria ter impedido
- ☐ (b) qual evidência a trilha deve conter para o pós-incidente

⑦ Armadilhas

- “O modelo é tão bom que não precisa de teste”**: o DORA 2024 mostrou exatamente o contrário — produtividade individual sem estabilidade de entrega é um custo escondido [9]
- Permissão ampla “só por enquanto”**: tokens com escopo global e diretórios liberados são o vetor favorito de incidentes [17]
- Autonomia total sem approval gates**: cancelar a confirmação humana “para acelerar” transfere o risco de erro para a escala — um erro repetido 100 vezes não é 100 vezes mais rápido, é 100 vezes mais caro [16]
- Sem observabilidade**: agente que faz muito e não deixa rastro é um passivo de auditoria ambulante [12]
- Comprar o hype do “agente pronto”**: o relatório da LangChain com mais de 1.300 profissionais mostra que 57% das organizações já têm agentes em produção — mas também que observabilidade e evals, as fundações do harness, ainda são os itens menos maduros [12]

Passo 2 — Anatomia de um Harness: O Corpo Que Carrega o Cérebro

Estágio: Arnês · **Origem:** Cap. 2 — Anatomia de um Harness: O Corpo Que Carrega o Cérebro

① Objetivo do passo

Dissecar as camadas do harness — ambiente de execução, ferramentas, memória, estado e loops de feedback — mostrando cada peça com exemplo concreto.

② Pré-requisito

Passo 1 concluído

③ Entregas

- `tests/test_contrato.py`

④ Execução

Dissecar as camadas do harness — ambiente de execu

```
"""Contrato de teste: 4 clausulas + caso de erro."""

def classificar(nome, escopo):
    if not nome or not escopo:
        return "desconhecida"
    return "permitida" if nome in {"ler", "buscar"} else "bloqueada"

casos = [("ler", "prod", "permitida"), ("apagar", "prod", "bloqueada"),
         ("ler", "", "desconhecida")]
for nome, esc, esperado in casos:
    ok = classificar(nome, esc) == esperado
    print(nome, esc, "->", classificar(nome, esc), "OK" if ok else "FALHA")
```

⑤ Verificação / Gate

```
python scripts/validar-codigo.py livros/harness-engineering --capitulo 2 --executar
```

⑥ Feito quando...

- ☐ Exercício 1 — Teste a âncora do capítulo.** Escreva testes parametrizados para a função de classificação abaixo
- ☐ O teste determinístico é a âncora que impede o agente de “funcionar” errando em silêncio
- ☐ Exercício 2 — Da observação ao caso de teste.** Pegue uma falha real que você já viu um agente cometer (uma resposta errada)
- ☐ Transforme-a em três camadas: (a) o caso de teste que teria capturado
- ☐ (b) o assert que descreve o contrato
- ☐ (c) o gate de CI onde ele roda
- ☐ A disciplina de traduzir observação em teste é o que transforma harness em prática contínua

⑦ Armadilhas

- Ambiente compartilhado “para simplificar”**: rodar o agente com as mesmas permissões do operador transforma qualquer erro em incidente de segurança; o isolamento é a primeira linha [7][17]
- Ferramentas ilimitadas**: cada ferramenta nova é superfície de ataque nova; comece com o mínimo e expanda sob demanda [13][14]
- Memória só no prompt**: se o estado vive apenas na janela de contexto, uma execução interrompida perde tudo; persista o que importa [6][18]
- Feedback só inferencial**: depender apenas de “o agente disse que funcionou” é aceitar a palavra do escalador sem conferir o mosquetão; testes determinísticos são a âncora [5][18]
- Guardrails “por convenção”**: pedir “por favor não apague” no prompt não é guardrail; o bloqueio precisa ser estrutural [16][20]

Passo 3 — Test Harness: A Herança da Engenharia de Software

Estágio: Arnês · **Origem:** Cap. 3 — Test Harness: A Herança da Engenharia de Software

① Objetivo do passo

Apresentar o harness de teste — fixtures, execução determinística, linters e CI — como a primeira linha de verificação do trabalho do agente.

② Pré-requisito

Passo 2 concluído

③ Entregas

- `evals/regua.py`

④ Execução

Apresentar o harness de teste — fixtures, execução

```
"""Regua de qualidade: 4 criterios explicita e repetivel."""

def pontuar(resposta, contexto):
    relev = 1.0 if contexto.lower() in resposta.lower() else 0.4
    compl = min(1.0, len(resposta) / 200.0)
    seg = 0.0 if "rm -rf" in resposta else 1.0
    rastr = 1.0 if "[ref]" in resposta else 0.3
    return (relev + compl + seg + rastr) / 4

boa = "O harness isola o agente [ref]. Custo cai, controle sobe."
ruim = "rm -rf /tmp"
print("boa:", round(pontuar(boa, "harness"), 2))
print("ruim:", round(pontuar(ruim, "harness"), 2))
```

⑤ Verificação / Gate

```
python scripts/validar-codigo.py livros/harness-engineering --capitulo 3 --executar
```

⑥ Feito quando...

- ☐ Exercício 1 — Crie sua régua de qualidade.** O capítulo mostrou a régua de referência
- ☐ Agora implemente uma versão mínima que pontua uma resposta do agente em quatro critérios: relevância
- ☐ O objetivo é tornar o julgamento explícito
- ☐ Exercício 2 — Defina seus critérios.** Para sua aplicação
- ☐ Liste quatro critérios de qualidade que um julgamento humano usaria
- ☐ Se um critério não puder ser automatizado

- ☐ Documente por quê — a régua honesta também sabe o que não mede

7 Armadilhas

- Confiar na autoavaliação do modelo**: “o agente disse que completou” não é evidência; é narrativa [18]
- Testar a elegância em vez do contrato**: avaliações de “qualidade” subjetivas não substituem asserções verificáveis [2]
- Golden tests frágeis**: esperar a saída exata de um sistema probabilístico quebra o teste; teste propriedades e estruturas, não strings exatas [3]
- Gate não bloqueante**: rodar testes “para relatório” sem bloquear a integração é decorativo; o gate precisa falhar o fluxo [1]
- Sem régua de benchmark**: avaliar o agente só em casos próprios esconde regressões; benchmarks públicos dão a régua objetiva [2][3]

Passo 4 — Safety Harness e Guardrails: A Camada Que Impede a Queda

Estágio: Arnês · **Origem:** Cap. 4 — Safety Harness e Guardrails: A Camada Que Impede a Queda

1 Objetivo do passo

Mostrar como proteger o sistema contra ações destrutivas do agente — aprovações humanas, limites de escopo, bloqueio de tool calls e princípio do menor privilégio.

2 Pré-requisito

Passo 3 concluído

3 Entregas

- `config/guardrails.yaml`

4 Execução

Mostrar como proteger o sistema contra ações destr

```
"""Guardrail fail-closed: zona declarada, nao negociada."""

def avaliar(tipo, alvo):
    regras = [("ler", {"*.md"})], ("buscar", {"web"})]
    for t, alvos in regras:
        if tipo == t and (alvo in alvos or alvo.endswith(".md")):
            return "PERMITIDA"
    return "BLOQUEADA (fail-closed)"
```

```
for acao in [("ler", "notas.md"), ("apagar", "dados")]:  
    print(acao, "->", avaliar(*acao))
```

⑤ Verificação / Gate

```
python scripts/validar-codigo.py livros/harness-engineering --capitulo 4 --executar
```

⑥ Feito quando...

- ☐ Exercício 1 — Classificador de ações com fallback seguro.** Implemente um guardrail mínimo com a filosofia fail-closed: se a classificação não reconhecer a ação
- ☐ A regra de ouro do safety harness é errar para o lado seguro — nunca “deixar passar porque não sei” [20]
- ☐ Exercício 2 — Quebra do guardrail.** No Exercício 1
- ☐ Adicione uma regra para bloqueá-la
- ☐ Esse exercício reproduz a classe de vulnerabilidades de traversal que a OWASP destaca [20]
- ☐ Exercício 3 — Política de exceção.** Defina o fluxo de exceção do seu guardrail: quem autoriza uma ação bloqueada
- ☐ Sem esse fluxo

⑦ Armadilhas

- Segurança no prompt**: “por favor, não apague nada” não é guardrail; é sugestão [20]
- Permitir por padrão**: bloquear só o que se conhece deixa o desconhecido livre; inverta para deny by default
- Approval gate sem trilha**: aprovação sem registro é indecifrável depois; registre quem, o quê e quando [16]
- Token global “por conveniência”**: o escopo amplo é o vetor favorito de incidentes; escope por tarefa [17]
- Confiar no caminho exibido**: symlinks e caminhos relativos podem mentir; resolva antes de decidir [16]

Passo 5 — O Ciclo ReAct e os Loops de Execução

Estágio: Corda · **Origem:** Cap. 5 — O Ciclo ReAct e os Loops de Execução

① Objetivo do passo

Construir o primeiro harness funcional: o loop Reason → Act → Observe com execução de ferramentas, tratamento de erro e iteração até o objetivo.

② Pré-requisito

Passo 4 concluído

③ Entregas

- `loop/reat.py`

④ Execução

Loop ReAct mínimo

```
"""Loop ReAct: propoe, executa, observa."""

class Modelo:
    def raciocinar(self, ctx):
        return "calc:2+3*4" if "quanto e" in ctx else "finalizar"

class Harness:
    def __init__(self): self.hist = []
    def calc(self, exp):
        return str(eval(exp, {"__builtins__": {}}, {}))
    def rodar(self, i):
        ctx = "instrucao: " + i
        for _ in range(5):
            acao = Modelo().raciocinar(ctx)
            if acao == "finalizar":
                return "objetivo atingido"
            obs = self.calc(acao.split(":", 1)[1])
            self.hist.append(obs)
            ctx += " | obs: " + obs

h = Harness()
print(h.rodar("Quanto e 2+3*4?"))
print(h.hist)
```

⑤ Verificação / Gate

```
python scripts/validar-codigo.py livros/harness-engineering --capitulo 5 --executar
```

⑥ Feito quando...

- ☐ Exercício 1 — Loop com ferramenta real.** Substitua a calculadora do `HarnessReAct` deste capítulo por uma ferramenta que lê um arquivo JSON de configuração
- ☐ A observação deve voltar estruturada: sucesso
- ☐ Exercício 2 — O ciclo completo.** Expanda o loop do capítulo para usar a ferramenta do Exercício 1
- ☐ Rode três execuções: uma com arquivo válido
- ☐ Registre o histórico
- ☐ Descreva como o agente corrigiria o curso em cada caso
- ☐ Exercício 3 — Limite

⑦ Armadilhas

- Chamada única “direta ao modelo”**: sem loop, sem observação, sem correção de curso — o agente é um LLM com prompt bonito [4]
- Retry infinito**: falha permanente vira gasto infinito; sempre limite + backoff [19]

- Erro tratado como dado**: registrar “503” como conteúdo da resposta faz o agente raciocinar sobre um erro como se fosse fato [5]
- Sem registro do histórico**: quando o agente erra, não há como saber o que ele viu; a trilha é a memória de auditoria [12]
- Executar sem guardrails**: o ponto de execução do loop deve ser o mesmo ponto de classificação de ações do Capítulo 4 — senão o loop foge do capacete [16][20]

Passo 6 — Sandboxes, Permissões e o Controle de Execução

Estágio: Corda · **Origem:** Cap. 6 — Sandboxes, Permissões e o Controle de Execução

① Objetivo do passo

Implementar isolamento real de execução — contêineres, permissões por escopo e políticas — para que o agente faça muito sem poder fazer qualquer coisa.

② Pré-requisito

Passo 5 concluído

③ Entregas

- `config/zonas.json`

④ Execução

Implementar isolamento real de execução — contêine

```
"""Tres zonas: segura, controlada, sensivel."""

ZONAS = {"segura": "executar", "controlada": "aplicar politica",
        "sensivel": "exigir humano"}

def rotear(nome, zona):
    if zona not in ZONAS:
        return "bloquear"
    return f"{nome}: {ZONAS[zona]}"

for acao in [("ler_dados", "segura"), ("deploy", "sensivel")]:
    print(rotear(*acao))
```

⑤ Verificação / Gate

```
python scripts/validar-codigo.py livros/harness-engineering --capitulo 6 --executar
```

⑥ Feito quando...

- ☐ Exercício 1 — Sandbox mínimo por política.** Implemente um sandbox conceitual que decide
- ☐ A separação política/execução é a lição central: o sandbox não decide o que é certo — ele aplica o que foi decidido [19]
- ☐ Exercício 2 — Inventário de superfície.** Liste as ferramentas do seu agente
- ☐ (b) exige ambiente real com aprovação
- ☐ (c) nunca deve ser oferecida ao agente
- ☐ Você terá a base do arquivo de política do seu harness
- ☐ Exercício 3 — Demonstração de dano.** Escolha uma ferramenta perigosa (por exemplo

⑦ Armadilhas

- Credenciais do operador**: o agente com as permissões de quem o invoca é o incidente mais previsível do harness [17]
- Sandbox de mentira**: restringir arquivos mas liberar rede (ou vice-versa) é contenção parcial; o escopo precisa cobrir todas as dimensões [19]
- Token global “para o agente fazer tudo”**: a conveniência de hoje é o vazamento de amanhã; escopo por tarefa [17]
- Trilha que ninguém lê**: log sem estrutura não é auditoria; eventos em JSON consultáveis é que são [12]
- Isolar depois**: adicionar a sandbox após o incidente é aprender no caro; a contenção entra na primeira versão [20]

Passo 7 — Gestão de Contexto: Combatendo o Context Rot

Estágio: Corda · **Origem:** Cap. 7 — Gestão de Contexto: Combatendo o Context Rot

① Objetivo do passo

Ensinar a manter o agente focado em tarefas longas — compactação, offloading de ferramentas, divulgação progressiva e o padrão do loop de longa duração.

② Pré-requisito

Passo 6 concluído

③ Entregas

- `trilha/trilha.py`

④ Execução

Ensinar a manter o agente focado em tarefas longas

```
"""Trilha estruturada: auditoria de execucao suspeita."""
```

```
class Trilha:
    def __init__(self): self.ev = []
    def registrar(self, passo, acao):
        self.ev.append({"passo": passo, "acao": acao})
    def auditar(self, esperadas):
        return [e for e in self.ev if e["acao"] not in esperadas]

t = Trilha()
for p, a in [(1, "buscar"), (2, "ler"), (3, "apagar")]:
    t.registrar(p, a)
print("desvios:", t.auditar({"buscar", "ler"}))
```

⑤ Verificação / Gate

```
python scripts/validar-codigo.py livros/harness-engineering --capitulo 7 --executar
```

⑥ Feito quando...

- ☐ Exercício 1 — Registro estruturado.** Implemente um logger que registra cada passo do agente em JSON estruturado: ação
- ☐ A trilha estruturada é o que torna o agente auditável — sem ela
- ☐ Exercício 2 — Caça à causa raiz.** Usando a trilha do Exercício 1
- ☐ Percorra os eventos
- ☐ Identifique a sequência exata de decisões que levou ao desvio —
- ☐ Exercício 3 — Métricas do arnês.** Defina três métricas para o seu harness (por exemplo: taxa de sucesso por tarefa
- ☐ Registre-as por uma semana

⑦ Armadilhas

- Jogar tudo no prompt**: o antídoto para “contexto pequeno” que envenena o contexto grande; divulgue progressivamente [6]
- Histórico bruto infinito**: cada observação fica para sempre, e a instrução se afoga; compacte por blocos [6]
- Estado só na conversa**: sem arquivos de progresso, uma interrupção ou um retry perde tudo; persista o estado [18]
- Revisão sem critérios**: “o agente disse que terminou” não é entrega verificada; critérios objetivos + loop de revisão [1]
- Loop de revisão infinito**: revisar sem limite é o novo retry infinito; limite + escalação [1][19]

Passo 8 — Harness em Produção: Observabilidade, Evals e o Engenheiro Agêntico

Estágio: Corda · **Origem:** Cap. 8 — Harness em Produção: Observabilidade, Evals e o Engenheiro Agêntico

① Objetivo do passo

Fechar a obra com a operação de harnesses em escala — observabilidade, evals como testes de regressão do agente e o novo papel do engenheiro que desenha ambientes.

② Pré-requisito

Passo 7 concluído

③ Entregas

- manifesto/harness.json

④ Execução

Fechar a obra com a operação de harnesses em escal

```
"""Manifesto declarativo + kill-switch."""

class Painei:
    def __init__(self): self.execs = []; self.pausado = False
    def registrar(self, e): self.execs.append(e)
    def kill_switch(self):
        self.pausado = True
        self.execs = []
    def status(self):
        return f"pausado={self.pausado} ativas={len(self.execs)}"

p = Painei()
p.registrar("run-1"); p.registrar("run-2")
print("antes:", p.status())
p.kill_switch()
print("depois:", p.status())
```

⑤ Verificação / Gate

```
python scripts/validar-codigo.py livros/harness-engineering --capitulo 8 --executar
```

⑥ Feito quando...

- ☐ Exercício 1 — Runbook de incidente.** Escreva um runbook de 10 passos para um incidente de agente em produção: detecção
- ☐ Inclua o comando de kill-switch (pausar execuções)
- ☐ Exercício 2 — Custo
- ☐ Documente o fluxo: quem aprova
- ☐ O limiar transforma o custo em controle — a recomendação de cancelar projetos sem retorno claro que a Gartner publicou é o avesso dessa disciplina [10]
- ☐ Exercício 3 — Plano de rollback.** Liste os artefatos que o agente pode modificar (arquivos)
- ☐ Se um artefato não tiver rollback

⑦ Armadilhas

- Observabilidade depois do incidente**: instrumentar a caixa preta quando ela já custou caro é o padrão mais caro do mercado; instrumente no primeiro dia [12]
- Eval de uma vez só**: medir o agente uma vez e nunca mais é tirar foto de quem precisa de exame periódico; eval é monitoramento de tendência [5]
- LLM-as-a-judge sem calibração**: o julgador tem viés; sem conferência humana periódica, o eval mede o viés do julgador [19]
- Deploy sem gate de regressão**: mudança que vai para produção sem check-up é aposta; bloqueie a regressão [12]
- Engenheiro que só escreve código**: no mundo agêntico, quem não desenha ambiente e especifica intenção vira gargalo — o harness é o produto do engenheiro [1]

Checklist Mestre

Passo 1 — A Revolução dos Agentes: Por Que o Modelo Não Basta

- ☐ Exercício 1 — Inventário do arnês.** Liste os cinco componentes de um harness (âncora
- ☐ Escreva uma frase dizendo qual falha do agente ele evita
- ☐ Use uma tabela como a abaixo
- ☐ Complete a função `executar_acao` para que o harness valide a ação antes de entregá-la ao modelo — a lição central do capítulo: quem executa é o harness
- ☐ Exercício 3 — Diagnóstico.** Um agente de suporte apagou um arquivo de produção porque o prompt do sistema dizia “você tem autonomia total”
- ☐ Aponte: (a) qual componente do harness deveria ter impedido
- ☐ (b) qual evidência a trilha deve conter para o pós-incidente

Passo 2 — Anatomia de um Harness: O Corpo Que Carrega o Cérebro

- ☐ Exercício 1 — Teste a âncora do capítulo.** Escreva testes parametrizados para a função de classificação abaixo
- ☐ O teste determinístico é a âncora que impede o agente de “funcionar” errando em silêncio
- ☐ Exercício 2 — Da observação ao caso de teste.** Pegue uma falha real que você já viu um agente cometer (uma resposta errada
- ☐ Transforme-a em três camadas: (a) o caso de teste que teria capturado
- ☐ (b) o assert que descreve o contrato
- ☐ (c) o gate de CI onde ele roda
- ☐ A disciplina de traduzir observação em teste é o que transforma harness em prática contínua

Passo 3 — Test Harness: A Herança da Engenharia de Software

- ☐ Exercício 1 — Crie sua régua de qualidade.** O capítulo mostrou a régua de referência
- ☐ Agora implemente uma versão mínima que pontua uma resposta do agente em quatro critérios: relevância
- ☐ O objetivo é tornar o julgamento explícito
- ☐ Exercício 2 — Defina seus critérios.** Para sua aplicação
- ☐ Liste quatro critérios de qualidade que um julgamento humano usaria
- ☐ Se um critério não puder ser automatizado
- ☐ Documente por quê — a régua honesta também sabe o que não mede

Passo 4 — Safety Harness e Guardrails: A Camada Que Impede a Queda

- ☐ Exercício 1 — Classificador de ações com fallback seguro.** Implemente um guardrail mínimo com a filosofia fail-closed: se a classificação não reconhecer a ação
- ☐ A regra de ouro do safety harness é errar para o lado seguro — nunca “deixar passar porque não sei” [20]
- ☐ Exercício 2 — Quebra do guardrail.** No Exercício 1
- ☐ Adicione uma regra para bloqueá-la
- ☐ Esse exercício reproduz a classe de vulnerabilidades de traversal que a OWASP destaca [20]
- ☐ Exercício 3 — Política de exceção.** Defina o fluxo de exceção do seu guardrail: quem autoriza uma ação bloqueada
- ☐ Sem esse fluxo

Passo 5 — O Ciclo ReAct e os Loops de Execução

- ☐ Exercício 1 — Loop com ferramenta real.** Substitua a calculadora do `HarnessReAct` deste capítulo por uma ferramenta que lê um arquivo JSON de configuração
- ☐ A observação deve voltar estruturada: sucesso
- ☐ Exercício 2 — O ciclo completo.** Expanda o loop do capítulo para usar a ferramenta do Exercício 1
- ☐ Rode três execuções: uma com arquivo válido
- ☐ Registre o histórico
- ☐ Descreva como o agente corrigiria o curso em cada caso
- ☐ Exercício 3 — Limite

Passo 6 — Sandboxes, Permissões e o Controle de Execução

- ☐ Exercício 1 — Sandbox mínimo por política.** Implemente um sandbox conceitual que decide
- ☐ A separação política/execução é a lição central: o sandbox não decide o que é certo — ele aplica o que foi decidido [19]
- ☐ Exercício 2 — Inventário de superfície.** Liste as ferramentas do seu agente
- ☐ (b) exige ambiente real com aprovação
- ☐ (c) nunca deve ser oferecida ao agente
- ☐ Você terá a base do arquivo de política do seu harness
- ☐ Exercício 3 — Demonstração de dano.** Escolha uma ferramenta perigosa (por exemplo

Passo 7 — Gestão de Contexto: Combatendo o Context Rot

- ☐ Exercício 1 — Registro estruturado.** Implemente um logger que registra cada passo do agente em JSON estruturado: ação
- ☐ A trilha estruturada é o que torna o agente auditável — sem ela
- ☐ Exercício 2 — Caça à causa raiz.** Usando a trilha do Exercício 1
- ☐ Percorra os eventos
- ☐ Identifique a sequência exata de decisões que levou ao desvio —
- ☐ Exercício 3 — Métricas do arnês.** Defina três métricas para o seu harness (por exemplo: taxa de sucesso por tarefa
- ☐ Registre-as por uma semana

Passo 8 — Harness em Produção: Observabilidade, Evals e o Engenheiro Agêntico

- ☐ Exercício 1 — Runbook de incidente.** Escreva um runbook de 10 passos para um incidente de agente em produção: detecção
- ☐ Inclua o comando de kill-switch (pausar execuções)
- ☐ Exercício 2 — Custo
- ☐ Documente o fluxo: quem aprova
- ☐ O limiar transforma o custo em controle — a recomendação de cancelar projetos sem retorno claro que a Gartner publicou é o avesso dessa disciplina [10]
- ☐ Exercício 3 — Plano de rollback.** Liste os artefatos que o agente pode modificar (arquivos
- ☐ Se um artefato não tiver rollback